1. Problem Title & Link

Title: LeetCode 125 : Valid Palindrome

Link: LeetCode 125 - Valid Palindrome

2. Problem Statement (Short Summary)

Given a string s, determine whether it is a palindrome.

Rules:

- Ignore uppercase/lowercase differences
- Ignore spaces and special characters
- Consider only letters and digits

A palindrome reads the same from:

left → right

right → left

In simple terms:

Clean the string and check whether it reads the same from both directions.

3. Examples (Input → Output)

Example 1

Input:

s = "A man, a plan, a canal: Panama"

Output:

true

Explanation:

After removing symbols and converting to lowercase:

"amanaplanacanalpanama"

Palindrome.

Example 2

Input:

s = "race a car"

Output:

false

Explanation:

Cleaned string:

"raceacar"

Not palindrome.

Example 3

Input:

s=" "

Output:

true

Explanation:

Empty string after removing symbols is considered a palindrome.

4. Constraints

1 <= s.length <= 2 * 10^5

String contains:

English letters

digits

spaces

symbols

Special restrictions:

- Ignore non-alphanumeric characters
- Ignore case sensitivity

5. Core Concept (Pattern / Topic)

Two Pointers

6. Thought Process (Step-by-Step Explanation)

Brute Force

Steps:

1. Remove all special characters
2. Convert to lowercase
3. Reverse string
4. Compare original and reversed

Time:

O(n)

Space:

O(n)

Extra string storage is required.

Optimized Thinking

Use two pointers:

- left starts from beginning
- right starts from end

Algorithm:

1. Skip invalid characters
2. Convert characters to lowercase
3. Compare both characters
4. If mismatch → return false
5. Continue moving inward

This avoids creating extra strings.

7. Visual / Intuition Diagram (ASCII Diagram)

Input:

"A man, a plan, a canal: Panama"

Process:

A man, a plan, a canal: Panama

↑ ↑

left right

Compare:

A == a

Move inward

m == m

Move inward

a == a

Continue until pointers cross.

Result:

true

8. Pseudocode (Language Independent)

left=0

right=n-1

```
while left<right:

   while left<right and invalid:
      left++

   while left<right and invalid:
      right--

   if lowercase(leftChar)
     !=
     lowercase(rightChar):

      return false

   left++
   right--

return true
```

9. Code Implementation

Python

```
class Solution:

   def isPalindrome(self, s):

      left = 0
      right = len(s)-1

      while left < right:

         while left < right and not s[left].isalnum():
            left += 1

         while left < right and not s[right].isalnum():
            right -= 1
```

```
        if s[left].lower() != s[right].lower():
            return False

        left += 1
        right -= 1

    return True
```

Java

```
class Solution {

   public boolean isPalindrome(String s) {

      int left=0;
      int right=s.length()-1;

      while(left<right){

         while(left<right &&
              !Character.isLetterOrDigit(
                    s.charAt(left))){

            left++;
         }

         while(left<right &&
               !Character.isLetterOrDigit(
                     s.charAt(right))){

            right--;
         }

         char leftChar=
               Character.toLowerCase(
                     s.charAt(left));

         char rightChar=
```

```
                Character.toLowerCase(
                    s.charAt(right));

            if(leftChar!=rightChar){
                return false;
            }

            left++;
            right--;
        }

        return true;
    }
}
```

10. Time & Space Complexity

Time Complexity:

O(n)

Reason:

Each character is visited at most once.

Space Complexity:

O(1)

Reason:

Uses only pointer variables.

11. Common Mistakes / Edge Cases

1. Forgetting lowercase conversion

Wrong:

A != a

2. Forgetting to ignore symbols

Wrong:

"," ":" "!"

3. Missing empty string case

" "

Returns:

true

4. Using extra string unnecessarily

12. Detailed Dry Run (Step-byStep Table)

Input:

"A man, a plan, a canal: Panama"

Left	Right	Left Char	Right Char	Action
0	29	A	a	Match
2	28	m	m	Match
3	27	a	a	Match
...	...	...	...	Continue
End	-	-	-	Return true

Output:

true

13. Common Use Cases (Real-Life / Interview)

Text processing:

- Ignore formatting differences

DNA sequence checking

Pattern recognition

Data validation systems

14. Common Traps (Important!)

1. Not skipping special characters
2. Comparing before converting case
3. Incorrect pointer updates
4. Infinite loop from missing increments

15. Builds To (Related LeetCode Problems)

Progression:

- LC 125 → Valid Palindrome
- LC 680 → Valid Palindrome II
- LC 5 → Longest Palindromic Substring
- LC 647 → Palindromic Substrings
- LC 234 → Palindrome Linked List

16. Alternate Approaches + Comparison

Approach	Time	Space	Notes
Clean + Reverse String	O(n)	O(n)	Simple
Stack	O(n)	O(n)	Extra memory
Two Pointers	O(n)	O(1)	Optimal

17. Why This Solution Works (Short Intuition)

The two-pointer approach compares valid characters from both ends simultaneously. Since a palindrome reads the same in both directions, every pair of corresponding characters must match.

18. Variations / Follow-Up Questions

1. Allow deleting one character
2. Find longest palindrome substring
3. Check palindrome in linked list
4. Count total palindromic substrings
5. Solve for streaming input data
6.